

Interview Lens

Team 1: Ayro S Escobar, Ben Andre Quidlat, Hadeed Sarfaraz, Shaun Kenan Moukam, Joshua Asteway

[GitHub Repository: https://github.com/<username>/3354-Team1](https://github.com/<username>/3354-Team1)

1. Project Title

Interview Lens is an AI interview platform for mastering behavioral interviews. Interview Lens is a web-based platform that allows for college students practice realistic interviews with an AI interviewer that will shoot feedback back to the user on their answers and tracking their improvement over time.

2. Delegation of Tasks

Team Member	Responsibilities
Ayro Escobar	Database schema design and endpoint logic used across the platform; class diagram and repository/data-layer portions of this report.
Ben Andre Quidlat	Backend integration — connecting endpoints and validating data flow; unit testing and CI/CD pipeline setup; requirements (functional and non-functional) sections and report assembly/submission.
Hadeed Sarfaraz	AI pipeline design and logic for behavioral interview scenarios; sequence diagrams for the interview and feedback use cases; process-model selection rationale.
Shaun Kenan Moukam	Frontend UI/UX components, routing, and public deployment; use case diagram and architectural design (MVC) sections of this report.
Joshua Asteway	Quality assurance and documentation — writing test plans and acceptance criteria for each functional requirement, performing user acceptance testing on every increment, verifying the non-functional requirement targets, and maintaining team documentation and meeting notes.

3. Assumptions

We are explicitly making the following assumptions. Assumptions A3, A4, and A6 are made-up regulatory rules, which we state per the deliverable instructions allowing invented government/country-based rules so that every non-functional requirement type can be covered.

- 1. The platform initially targets college students in the United States and supports English-language interviews only.
- 2. Because our users are students and the platform may be utilized by universities, interview recordings, transcripts, and feedback are meant to be treated as educational records and handled in a FERPA-aligned manner.
- 3. We assume a U.S. "AI Interview Transparency Act of 2025" that requires any platform giving AI-generated hiring or interview feedback to clearly disclose that the feedback is AI-generated and to let users flag feedback for human review.

- **A5.** University single sign-on is not required for the first release so email-and-password authentication is sufficient.

4. Addressing Project Proposal Feedback

The feedback we received on our project proposal was that it was completed and sound, with no corrective changes requested.

5. Software Process Model

We are employing the incremental development process model

We chose incremental development for four reasons: our summer timeline is short, meaning that we must deliver a usable core system early, requirements for an AI product are inherently volatile meaning we can't predict how the AI interviewer should behave until it gets rolled out, each increment step gives us a model that we can present to the instructor/TA, so feedback we receive can shape the next step, and finally because the model fits the team structure and tooling with CI/CD pipeline validating every increment automatically.

We rejected the waterfall model because our requirements this is an evolving project where we interact with the AI interviewer, and we rejected pure integration-and-configuration because the core of our product must be custom-built.

6. Software Requirements

6.1 Functional Requirements

ID	Functional Requirement
FR-1	The system shall allow users to register an account with name, email, and password, and to log in and log out. Duplicate emails shall be rejected.
FR-2	The system shall allow a logged-in student to start a mock behavioral interview session by selecting a target job role and a difficulty level.
FR-3	The system shall use the AI engine to generate at least five behavioral interview questions per session, including adaptive follow-up questions based on the student's previous answer.
FR-4	The system shall accept student answers as typed text or recorded audio, and shall transcribe audio answers to text before analysis.
FR-5	Upon session completion, the system shall generate a feedback report that scores each answer on STAR structure, clarity, and confidence, and gives an overall session score from 0 to 100 with written improvement suggestions.
FR-6	The system shall store every completed session and its report, and shall display the student's history and score trends over time on a progress dashboard.
FR-7	The system shall allow an administrator to add, edit, and remove questions and question categories in the question bank.

6.2 Non-Functional Requirements

ID	Type	Non-Functional Requirement
NFR-1	Usability	A first-time user shall be able to create an account and start their first mock interview in under 3 minutes without training or documentation; contextual help shall be available on every screen.
NFR-2	Performance	The system shall deliver the next interview question within 2 seconds and the full end-of-session feedback report within 15 seconds, for 95% of requests under normal load.
NFR-3	Space	The initial client download (web bundle) shall not exceed 5 MB, and a completed session (transcript plus report) shall consume no more than 2 MB of database storage on average.
NFR-4	Dependability	The platform shall maintain at least 99.5% monthly availability, and every answer shall be autosaved as it is submitted so that a crash or disconnect loses at most the answer currently in progress.
NFR-5	Security	All traffic shall use TLS 1.2 or higher; passwords shall be stored only as bcrypt hashes; session tokens shall expire within 24 hours; and role-based access control shall prevent students from accessing admin functions or other students' data.
NFR-6	Environmental	The system shall run in the two most recent versions of Chrome, Firefox, Safari, and Edge on desktop and mobile without any local installation, and shall be deployable on commodity Linux cloud instances.
NFR-7	Operational	The system shall support at least 500 concurrent interview sessions during peak recruiting season, and administrators shall be able to update the question bank without taking the platform offline.
NFR-8	Development	All code shall be written in TypeScript, held in the team GitHub repository, merged only through reviewed pull requests, and covered by unit tests at a minimum of 70% line coverage enforced by the CI/CD pipeline.
NFR-9	Regulatory	Per assumption A2, interview transcripts, recordings, and feedback shall be handled as FERPA-aligned educational records: private by default and never shared with third parties without the student's explicit consent.
NFR-10	Ethical	The platform shall clearly label all feedback as AI-generated, shall not sell user data, shall not use student responses to train AI models without explicit opt-in consent, and shall review AI prompts for bias against protected groups.
NFR-11	Accounting	Per assumption A6, the system shall keep an audit log of all account events and data-access events (logins, deletions,

ID	Type	Non-Functional Requirement
		admin changes) and retain these logs for 12 months for audit purposes.
NFR-12	Safety / Security (Legislative)	Per assumptions A3 and A4, the system shall comply with the assumed AI Interview Transparency Act of 2025 (disclosure of AI-generated feedback and a human-review flag on every report) and shall permanently delete all of a user's data within 30 days of a deletion request.

7. Use Case Diagram

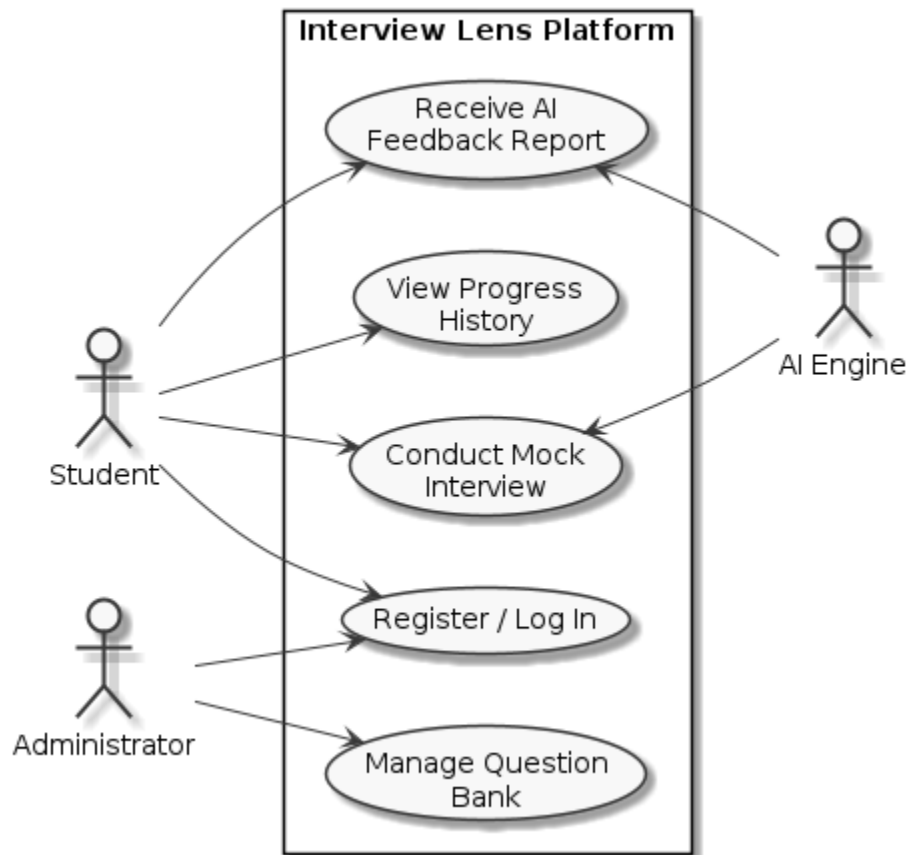


Figure 1. Use case diagram for Interview Lens

8. Sequence Diagrams

8.1 Register / Log In

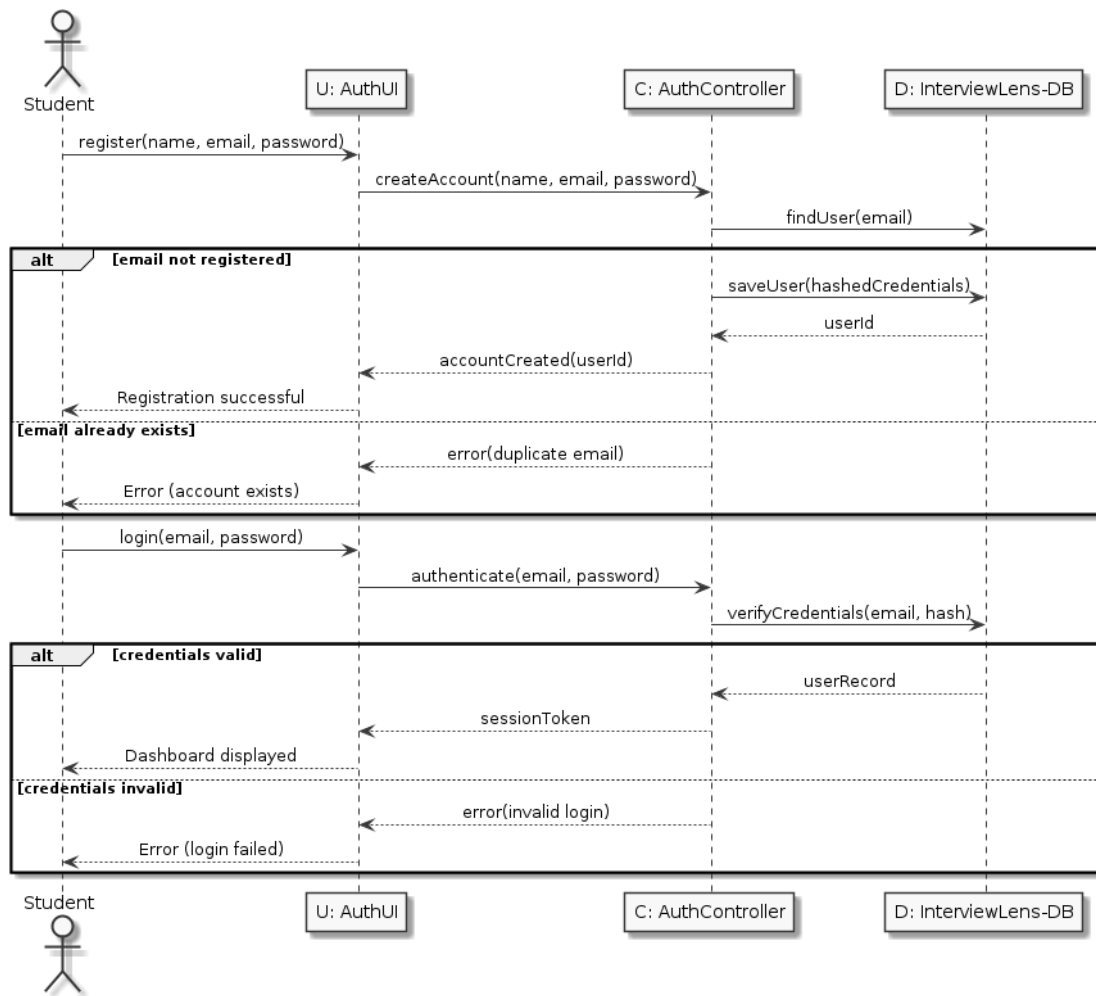


Figure 2. Sequence diagram for Register / Log In

8.2 Conduct Mock Interview

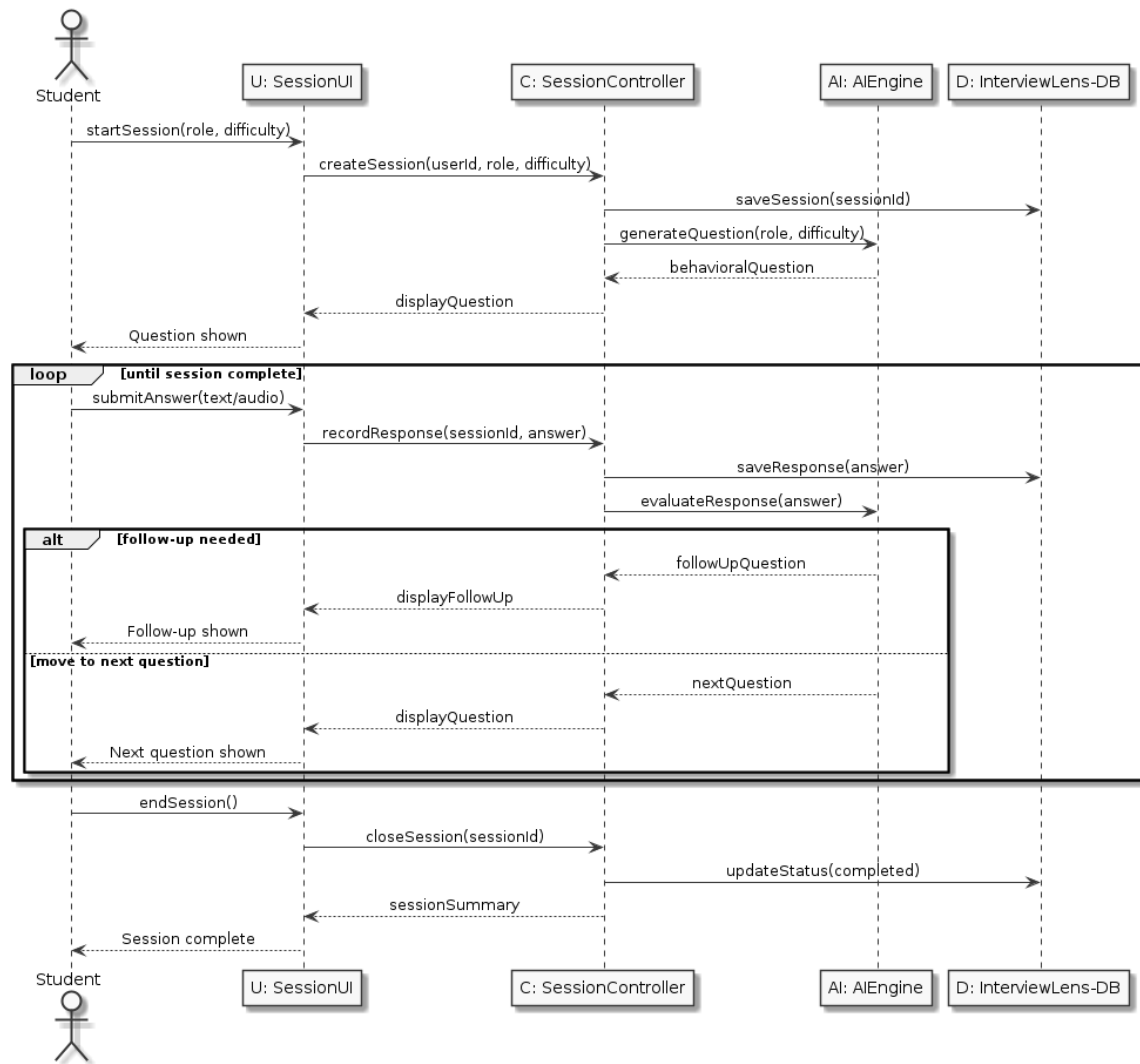


Figure 3. Sequence diagram for Conduct Mock Interview

8.3 Receive AI Feedback Report

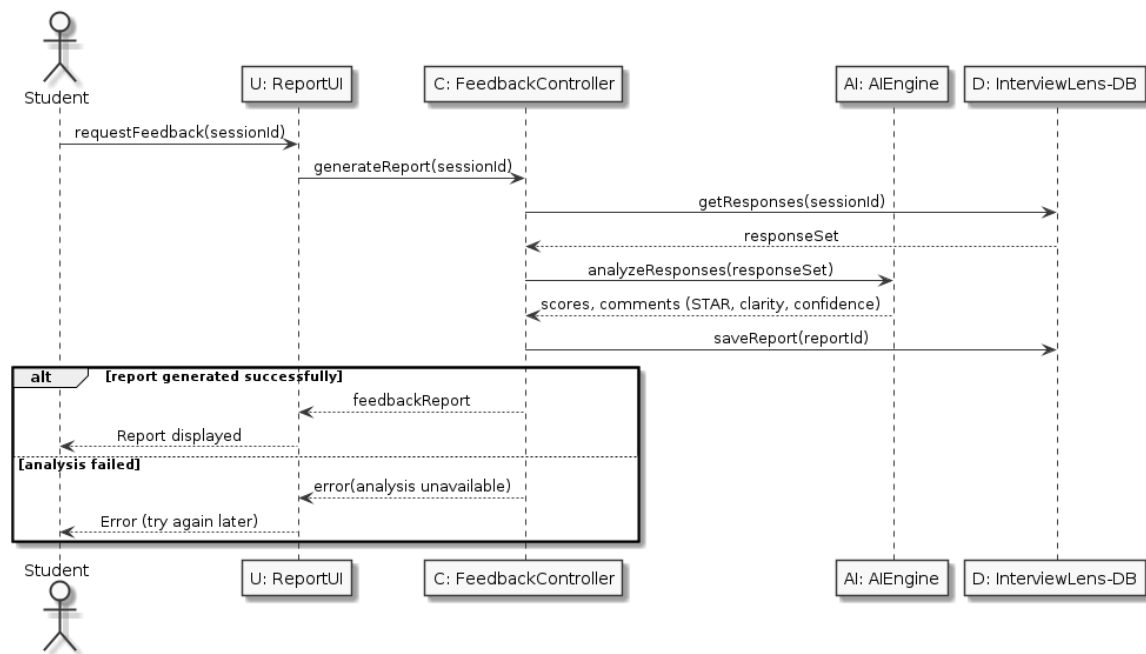


Figure 4. Sequence diagram for Receive AI Feedback Report

8.4 View Progress History

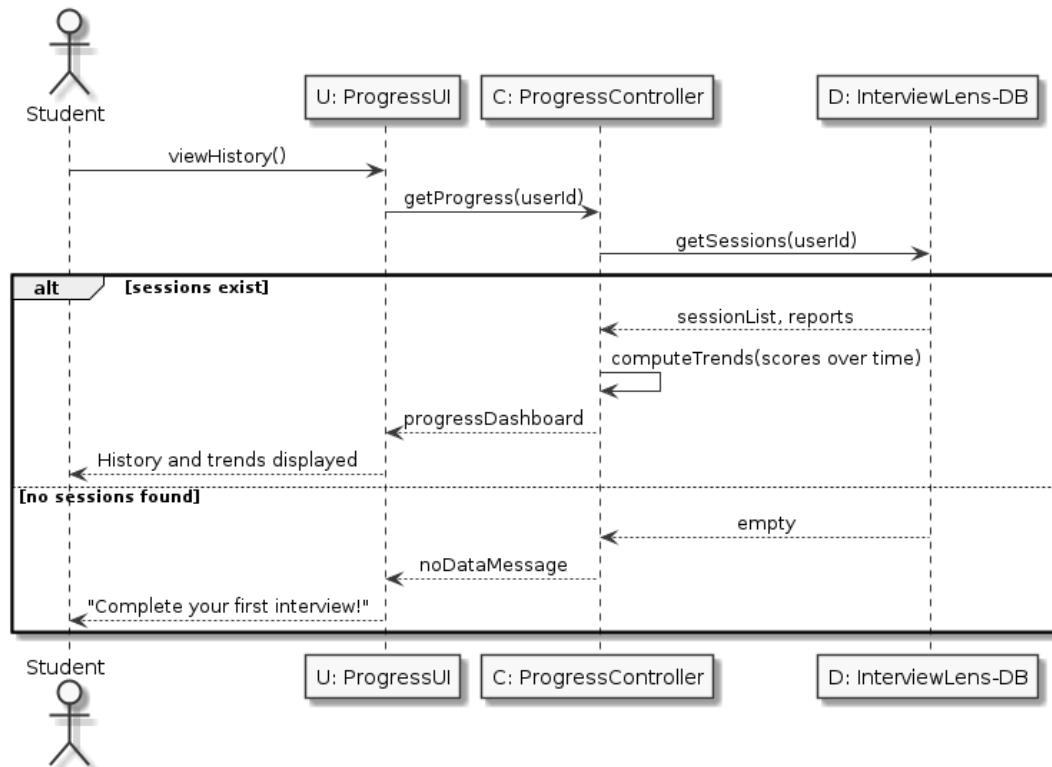


Figure 5. Sequence diagram for View Progress History

8.5 Manage Question Bank

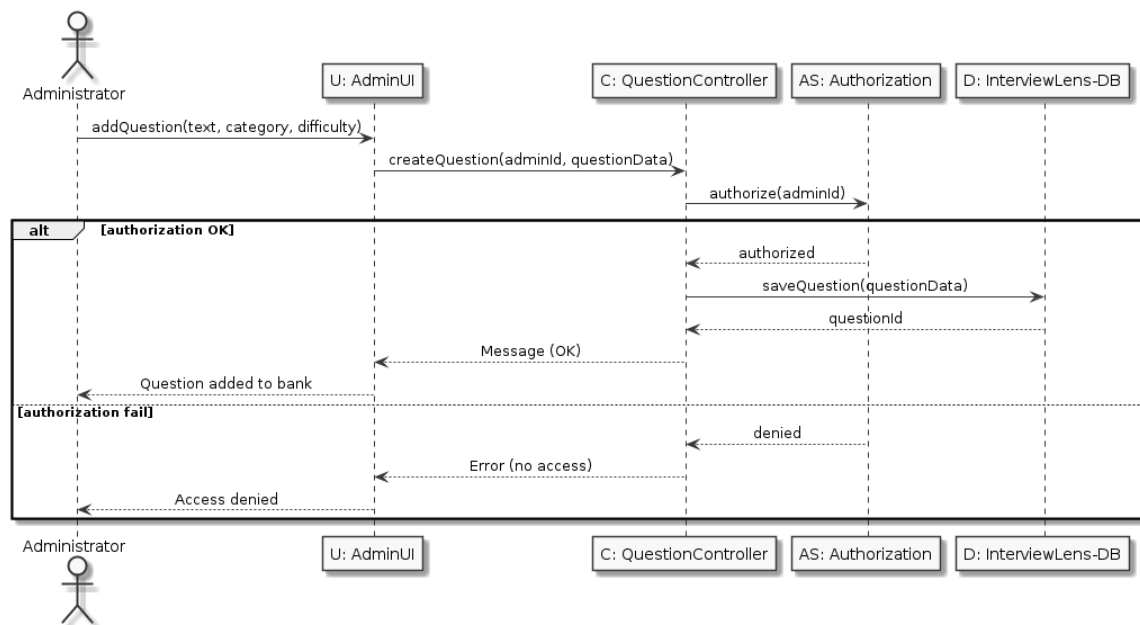


Figure 6. Sequence diagram for Manage Question Bank

9. Class Diagram

The single class diagram below includes all classes of the system. Student and Administrator specialize the abstract User class (generalization). An InterviewSession aggregates the Questions asked in it and composes the Responses recorded in it; the QuestionBank aggregates the reusable Question pool. Cardinalities are shown on every association: for example, one Student conducts zero or more InterviewSessions, and each completed session is evaluated by at most one FeedbackReport generated by the AIEngine.

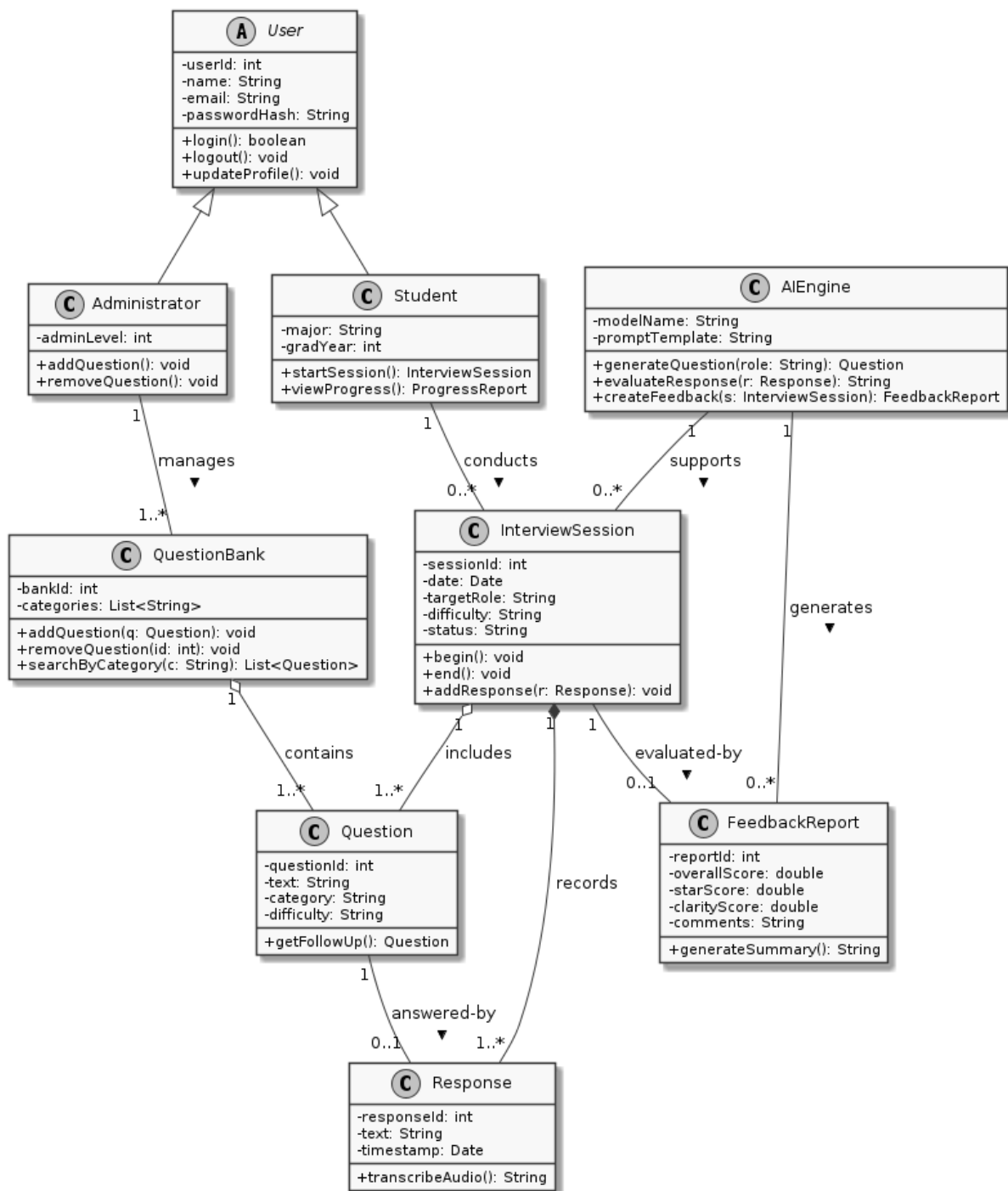


Figure 7. Class diagram for Interview Lens

10. Architectural Design

We chose the Model-View-Controller (MVC) architectural pattern (Sommerville, Figure 6.6). Interview Lens is an interactive web application in which the same underlying data — sessions, responses, and feedback — is presented in multiple ways (a live interview screen, a feedback report, a progress dashboard), and presentation is expected to change as we iterate on UI/UX. MVC isolates those presentations (View: React components) from the interaction logic (Controller: HTTP request processing, validation, and session/feedback/auth logic) and from the business logic and data (Model: domain classes, AI Engine calls, and the PostgreSQL database). This separation also maps cleanly onto our team

delegation: frontend work stays in the View, endpoint work in the Controller, and database plus AI pipeline work in the Model, letting all four members work in parallel with minimal conflicts.

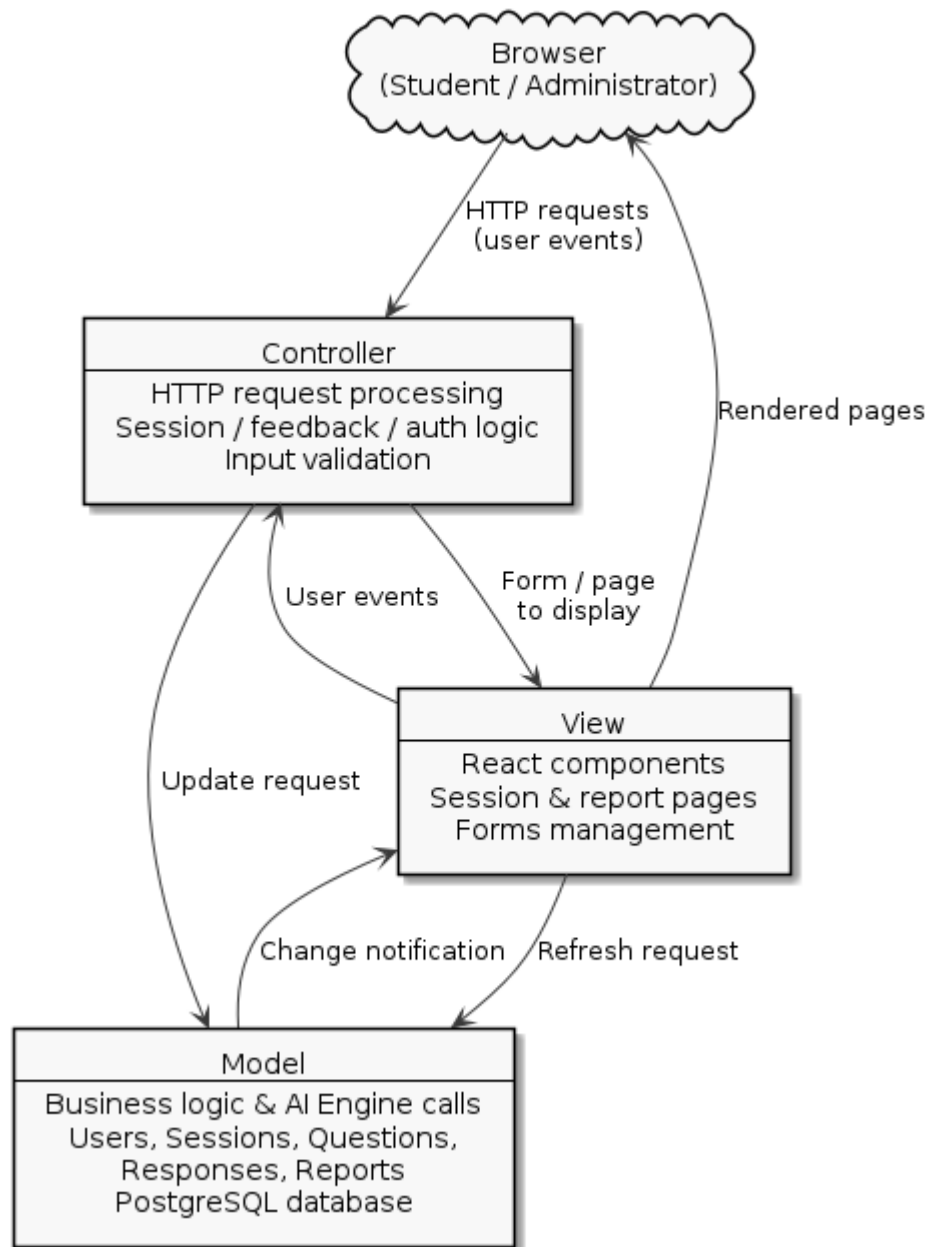


Figure 8. MVC architecture of Interview Lens